

Universidade Federal do Rio Grande do Sul
UFRGS

Instituto de Informática
INF



CMP270
Introduction To High Performance Computing

Comparação de Desempenho entre CUDA e OpenCL no PCAD

Relatório de Atividades

Autores

Lucas Villela Canôas
lucas.villela@inf.ufrgs.br

Docentes

Lucas Mello Schnorr
Philippe Oliver Alexandre Navaux

Porto Alegre, Agosto de 2024

Conteúdo

1	Introdução	2
2	Definição de Perímetro	3
3	Descrição do experimento	4
4	Desenvolvimento	5
5	Resultados Obtidos	6
5.1	Sem Controle de Clock	6
5.1.1	Tabelas Blaise CUDA	7
5.1.2	Tabelas Blaise OpenCL	8
5.1.3	Tabelas Sirius OpenCL	9
5.2	Com controle de Clock	10
5.2.1	Tabelas tempo de execução CUDA	12
5.2.2	Tabelas tempo de execução OpenCL	14
6	Análise PTX	17
6.1	Conclusões	19

1 Introdução

Neste relatório será tratado sobre as atividades realizadas durante a matéria de CMP270, no INF/UFRGS, tratando da análise de desempenho de simulação de ondas acústicas utilizando a infraestrutura do PCAD para a realização dos experimentos, com programação em CUDA e OpenCL para desenvolvimento.

As ondas acústicas tem importância no mapeamento de solo marinho, na análise de ondas sísmicas, arquitetura, exploração de recursos naturais de gás e petróleo, portanto a importância de analisar esse fenômeno e a possibilidade de simular um ambiente permite que tenhamos a previsibilidade do que pode ocorrer em determinadas situações.

As fórmulas foram conferidas utilizando principalmente a dissertação de mestrado do Rodrigo Fernandes, que realizou um trabalho parecido utilizando CUDA[4].

Para o pulso de ricker, foi utilizado:

$$(2 \cdot \pi \cdot f_0 \cdot (t_1 - t_0) \cdot \pi \cdot f_0 \cdot (t_1 - t_0) - 1) \cdot \exp(-\pi \cdot f_0 \cdot (t_1 - t_0) \cdot \pi \cdot f_0 \cdot (t_1 - t_0))$$

Para a definição inicial da onda:

$$\exp(-0.2 \cdot ((x - x_0)^2 + (y - y_0)^2))$$

Para o cálculo da onda:

$$\alpha = \frac{c^2 \cdot \Delta t^2}{\Delta h^2}$$

2 Definição de Perímetro

Analisar o desempenho de software desenvolvidos em CUDA e OpenCL em diversos dispositivos disponíveis no PCAD, medindo o tempo de execução e comparando para descobrir qual a melhor API para desenvolvimento de soluções paralelizáveis.

Os dispositivos utilizados foram:

- NVIDIA Tesla P100, Pascal, 4 x 3584 CUDA cores (Blaise)
- AMD Radeon RX 7900 XT, 84 cores (Sirius)
- NVIDIA RTX 4090, Ada (Tupi)
- NVIDIA Tesla K20m, Kepler, 2496 CUDA cores (Draco)
- NVIDIA GeForce GTX 1080Ti, 2 x 3584 CUDA cores (Beagle)
- NVIDIA RTX 4070, Ada, 5888 CUDA cores (Poti)

3 Descrição do experimento

Foi utilizado a simulação de ondas 2D em 1024x1024 pontos, realizando 36 execuções com medição de tempo, extração de consumo de memória GDDR, GPU, registro do clock da memória e da GPU.

A execução do experimento foi feita em 10 diferentes números de threads sendo: 32, 64, 128, 192, 256, 288, 320, 384, 480, 512. De modo a descobrir como este parâmetro pode interferir em cada API. A definição do número de threads por bloco (CUDA) e também por grupo de trabalho (OpenCL), sendo M a dimensão de 1024 pontos.

$$(M + threads - 1)/threads$$

Os experimentos deste trabalho utilizaram os recursos da infraestrutura PCAD, <http://gppd-hpc.inf.ufrgs.br>, no INF/UFRGS.

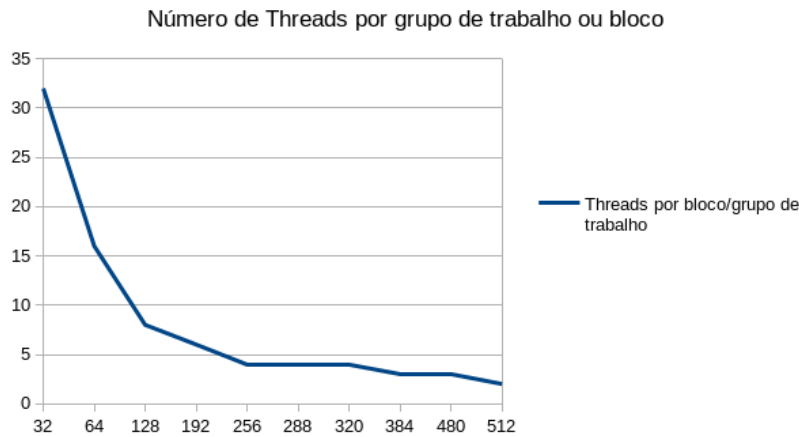


Figura 1: Variação do número de threads. Fonte: o autor

No momento do experimento, diversos computadores do PCAD não tinham a opção de fixar o clock da memória e da GPU, o que prejudicaria a reprodutibilidade, nesse sentido, apesar de testar as placas citadas, a única que oferece o rigor necessário é a máquina Blaise.

O código executado pode ser encontrado no [github\[2\]](#).

4 Desenvolvimento

Para a realização do experimento, foi desenvolvido um simulador de ondas 2D em CUDA e em OpenCL, com o objetivo de executar em ambientes Nvidia e AMD.

A simulação utiliza o método de diferenças finitas, para que não seja calcular a onda como algo contínuo, que iria utilizar muito tempo computacional.

Sendo assim, o desenvolvimento em CUDA e OpenCL para simulação de ondas, tem quatro kernels diferentes, que paralelizam o calculo da onda, a definição inicial e a definição de valores iniciais da malha a ser calculada.

CUDA tem uma sintaxe mais limpa e com poucos parâmetros passados para a sua execução, enquanto o OpenCL é mais verboso, isso se deve também pela possibilidade de execução do OpenCL em diversos dispositivos.

Para o desenvolvimento foi utilizado o código[1] como ponto de partida, porém o código é em 3D e a possibilidade de avaliar a corretude da implementação não foi viável por diversos motivos, então utilizando a ideia construída, tentou-se implementar em CUDA a simulação de ondas em 2D.

A ideia inicial do projeto, era utilizar apenas o código CUDA, mesmo em plataformas AMD, porém o projeto ZLUDA[5] que permitia a utilização de CUDA em AMD foi descontinuado. Nesse sentido, foi utilizado o código CUDA como base para a criação do código em OpenCL[3].

Para a execução do projeto, foi realizado dois scripts em Bash, que realiza o experimento em si e outro que realiza a execução nas máquinas do PCAD[2], as máquinas que não tiveram os dados aqui exibidos, tiveram problemas técnicos: impossibilidade de executar CUDA ou OpenCL por motivos variados.

No próprio código, a definição de frequência de clock da memória e da GPU foi definida, utilizando o hostname como meio de identificação da máquina e também a coleta dos demais dados necessários.

Para realizar a execução do experimento:

```
./slurm.sh cuda  
./slurm.sh opencl
```

5 Resultados Obtidos

5.1 Sem Controle de Clock

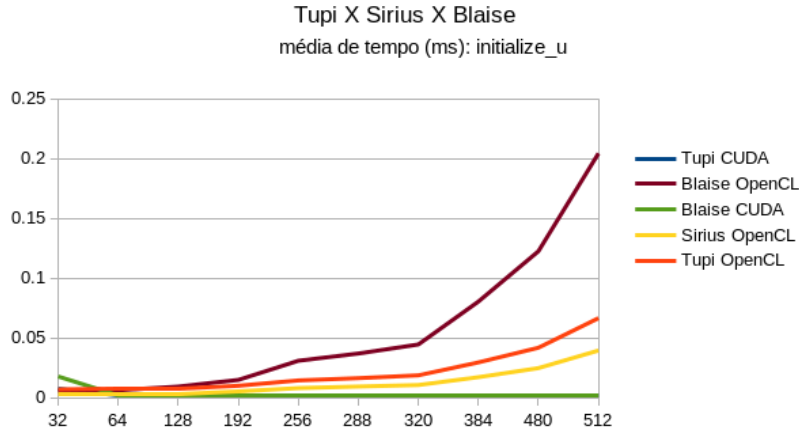


Figura 2: Tempo de execução do kernel initialize_u. Fonte: O autor

O kernel de inicialização da onda tem três blocos de memória, para que seja possível armazenar a onda anterior, a atual e a próxima, portanto este kernel é executado três vezes para definir os valores. Mesmo sendo uma operação simples, o OpenCL não tem um bom desempenho quando existem muitos threads e poucos threads por grupo de trabalho, mas mesmo em uma condição adversa para o OpenCL, é possível notar que o desempenho da Sirius é superior a da Tupi neste kernel.

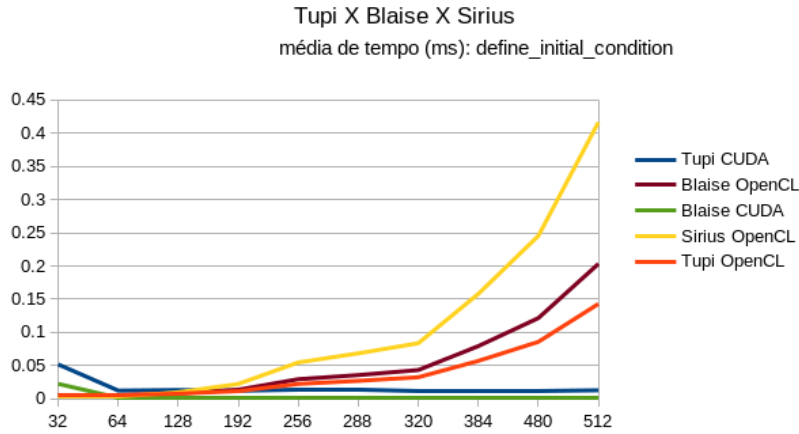


Figura 3: Definição de condição inicial da onda. Fonte: O autor

O pior tempo de execução neste Kernel, que não tem tantas operações, é realizado pelo OpenCL com 512 threads, nas máquinas Sirius, Blaise e Tupi e os melhores tempos foram feitos pela Sirius executando OpenCL com 32 Threads, superando até mesmo a Tupi. Porém nota-se a diferença entre o OpenCL e o CUDA em placas da Nvidia, pelo motivo de que o CUDA é construído para ser executado em hardware específico.

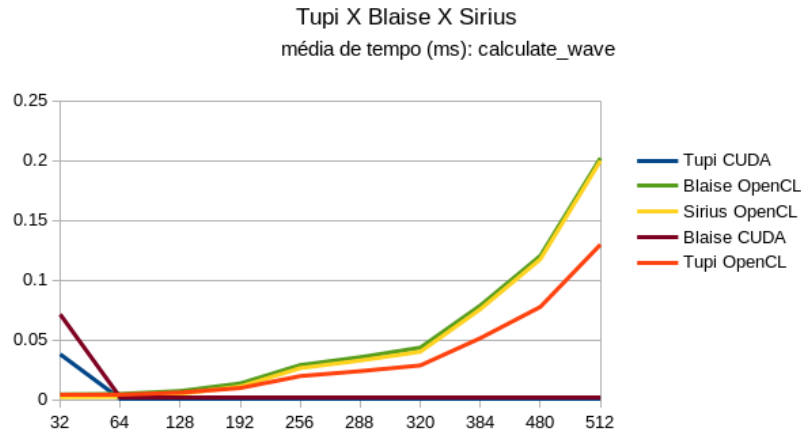


Figura 4: Tempo de execução do kernel calculate_wave. Fonte: O autor

Neste kernel que tem o maior número de operações, pode-se observar que a Sirius tem um desempenho significativamente maior do que a Tupi quando comparamos com os melhores tempos, tendo um resultado equiaparado ao Blaise em CUDA.

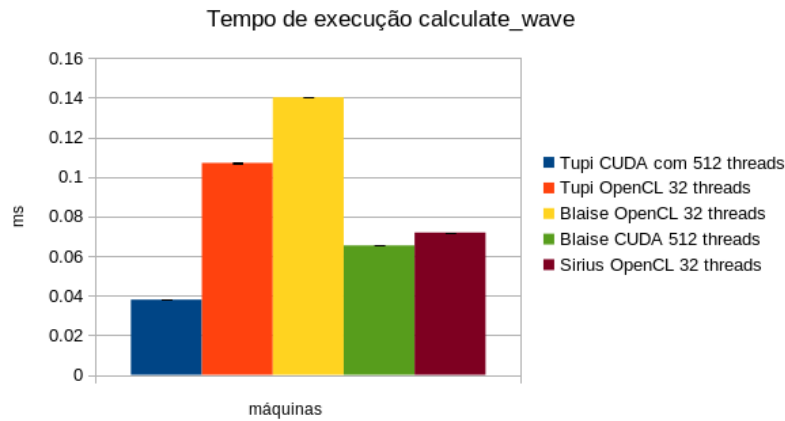


Figura 5: Tempo de execução do kernel calculate_wave. Fonte: O autor

Observando o tempo de execução do kernel que realiza o cálculo da onda, podemos notar o tempo de execução na Tupi o OpenCL é 2.87 vezes mais lento do que o CUDA e 2.14 vezes mais lento na Blaise. Comparando a Tupi com a Sirius, que tem fabricantes diferentes, mas placas que são equivalentes no mercado, temos um desempenho do OpenCL no fabricante da AMD que é 24% mais rápido, porém o CUDA ainda tem um tempo de execução menor da Tupi, alcançando 48% menos tempo do que a Sirius.

5.1.1 Tabelas Blaise CUDA

Threads	Soma do Tempo	Média do Tempo	Desvio Padrão
32	0.554592	0.0178900645	0.0007934733
64	0.066016	0.0021295483	0.0001236931
128	0.06496	0.0020954838	0.00007874
192	0.062848	0.0020949333	0.0000734846
256	0.064512	0.0020810322	0.0000888819
288	0.0648	0.0020903225	0.0000953939
320	0.069216	0.0022327741	0.0007648529
384	0.064352	0.0020758709	0.000041231
480	0.065888	0.0021254193	0.000125698
512	0.080608	0.002600258	0.0027347029

Tabela 1: Tabela de Resultados Blaise CUDA: initialize_u

Threads	Soma do Tempo	Média do Tempo	Desvio Padrão
32	0.698176	0.0225218064	0.0008894942
64	0.064992	0.0020965161	0.0000932737
128	0.064608	0.002084129	0.0000824621
192	0.062528	0.0020842666	0.00007
256	0.064416	0.0020779354	0.0000883176
288	0.064384	0.0020769032	0.0000632455
320	0.06464	0.0020851612	0.0000916515
384	0.065184	0.0021027096	0.0001148912
480	0.064672	0.0020861935	0.0000616441
512	0.064448	0.0020789677	0.0000916515

Tabela 2: Tabela de Resultados Blaise CUDA: define_initial_condition

Threads	Soma do Tempo	Média do Tempo	Desvio Padrão
32	2.215751	0.0714758387	0.0001868154
64	0.065037	0.0020979677	0.0000519615
128	0.065346	0.0021079354	0.0000519615
192	0.063077	0.0021025666	0.00005
256	0.065563	0.0021149354	0.00005
288	0.065642	0.0021174838	0.0000608276
320	0.06569	0.0021190322	0.0000519615
384	0.065215	0.0021037096	0.0000583095
480	0.065346	0.0021079354	0.0000591607
512	0.065322	0.0021071612	0.0000479583

Tabela 3: Tabela de Resultados Blaise CUDA: calculate_wave

5.1.2 Tabelas Blaise OpenCL

Threads	Soma do Tempo	Média do Tempo	Desvio Padrão
32	0.1848	0.0059612903	0.0003419751
64	0.199456	0.0064340645	0.0003909846
128	0.293568	0.0094699354	0.0030218917
192	0.466688	0.0150544516	0.0003510921
256	0.962368	0.031044129	0.0031027359
288	1.149856	0.037092129	0.0004440557
320	1.383648	0.0446338064	0.0004378891
384	2.495936	0.0805140645	0.0017565702
480	3.798976	0.1225476129	0.0021879848
512	6.334944	0.2043530322	0.0015809024

Tabela 4: Tabela de Resultados Blaise OpenCL: initialize_u

Threads	Soma do Tempo	Média do Tempo	Desvio Padrão
32	0.144352	0.0046565161	0.0001282245
64	0.156928	0.0050621935	0.0001275914
128	0.25008	0.0080670967	0.0025623389
192	0.428096	0.0138095483	0.0002947273
256	0.917472	0.0295958709	0.0018326043
288	1.110144	0.0358110967	0.0003159856
320	1.345248	0.0433950967	0.0003266766
384	2.461376	0.0793992258	0.001705424
480	3.768992	0.121580387	0.0021634503
512	6.298336	0.203172129	0.0013708589

Tabela 5: Tabela de Resultados Blaise OpenCL: define_initial_condition

Threads	Soma do Tempo	Média do Tempo	Desvio Padrão
32	0.1401768394	0.0045218335	0.0000298829
64	0.153896677	0.0049644089	0.0000266083
128	0.2285306477	0.0073719563	0.0000421881
192	0.42515973	0.01371483	0.0000334712
256	0.902932234	0.0291268462	0.0000410041
288	1.106305857	0.0356872857	0.0000791929
320	1.3528648398	0.0436408012	0.0000394311
384	2.4459545549	0.0789017598	0.0000394142
480	3.7346449684	0.1204724183	0.0000397413
512	6.2681618778	0.2021987702	0.0000539352

Tabela 6: Tabela de Resultados Blaise OpenCL: calculate_wave

5.1.3 Tabelas Sirius OpenCL

Threads	Soma do Tempo	Média do Tempo	Desvio Padrão
32	0.09704	0.0031303225	0.0003908591
64	0.1148	0.0037032258	0.0004636766
128	0.11488	0.0037058064	0.0005571863
192	0.16776	0.0054116129	0.0003229211
256	0.25392	0.0081909677	0.00030644
288	0.2948	0.0095096774	0.0003491508
320	0.33368	0.0107638709	0.0002883286
384	0.5372	0.0173290322	0.0002654679
480	0.77204	0.0249045161	0.0002794792
512	1.22828	0.0396219354	0.0003990748

Tabela 7: Tabela de Resultados Blaise OpenCL: initialize_u

Threads	Soma do Tempo	Média do Tempo	Desvio Padrão
32	0.11932	0.0038490322	0.0002924794
64	0.12568	0.0040541935	0.0002899696
128	0.30652	0.0098877419	0.0002758998
192	0.69652	0.022468387	0.0003471087
256	1.699167	0.0548118387	0.0003146683
288	2.123847	0.0685111935	0.000308181
320	2.597855	0.0838017741	0.0003056836
384	4.899195	0.1580385483	0.0003135549
480	7.607448	0.2454015483	0.0004658757
512	12.904333	0.4162688064	0.0003397985

Tabela 8: Tabela de Resultados Blaise OpenCL: define_initial_condition

Threads	Soma do Tempo	Média do Tempo	Desvio Padrão
32	0.0718316644	0.0023171504	0.0000127792
64	0.0787097038	0.0025390227	0.0000031457
128	0.1556636163	0.0050214069	0.000002748
192	0.3448840333	0.0111252913	0.0000037705
256	0.8228922114	0.02654491	0.0000347214
288	1.0205081084	0.0329196164	0.0000112147
320	1.2483179552	0.0402683211	0.0000145444
384	2.3500167537	0.075806992	0.0000668322
480	3.6466091293	0.1176325525	0.0000969532
512	6.1953853567	0.1998511405	0.0000773876

Tabela 9: Tabela de Resultados Blaise OpenCL: calculate_wave

5.2 Com controle de Clock

O único dispositivo com a possibilidade de controle de Clock no momento do experimento e com sucesso de execução de CUDA e OpenCL, foi a máquina Blaise. Nesse sentido, podemos observar melhor o comportamento de execução dessas duas ferramentas para a execução do simulador de ondas 2D em uma placa Nvidia. É importante destacar que o OpenCL tem o propósito de executar no máximo de dispositivos possível,

enquanto CUDA é desenvolvido especificamente para ser executado em placas Nvidia.

O kernel `initialize_u` é responsável por definir os valores zerados, as medições são muito parecidas entre si, inclusive com o kernel `define_c`, portanto a opção de construir apenas um gráfico.

O tempo de execução do OpenCL e CUDA, mesmo quando em situações muito favoráveis, tem uma diferença significativa, resultando em uma clara vantagem do CUDA, especialmente por estar em uma placa Nvidia.

Foi utilizado o CUDA 11.8 e o g++ 8.3.0, os resultados estão publicados no github[2].

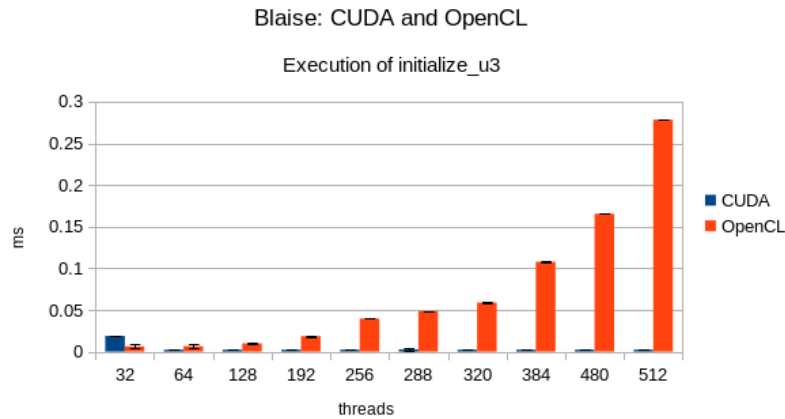


Figura 6: Definição da próxima onda como zero. Fonte: O autor

O tempo de execução para definir as condições iniciais, que contêm algumas operações matemáticas, já é significativamente mais alto.

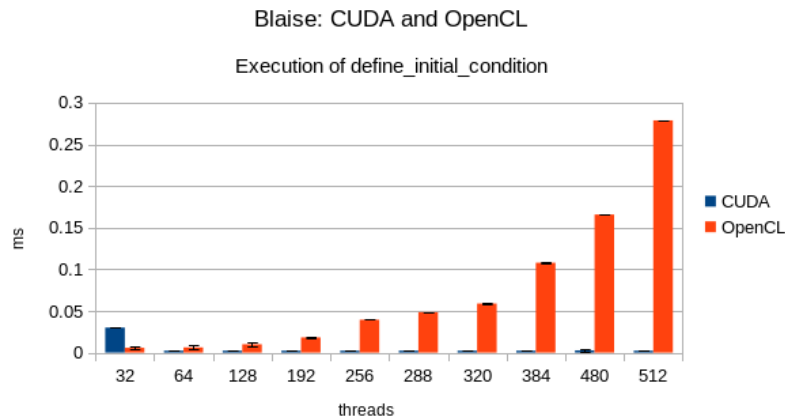


Figura 7: Tempo de execução do `define_initial_condition`. Fonte: O autor

No kernel `calculate_wave`, que tem muitas operações matemáticas sendo feitas em mais de um milhão de pontos, nos oferece uma observação notória da vantagem do CUDA, sendo 57% mais rápido.

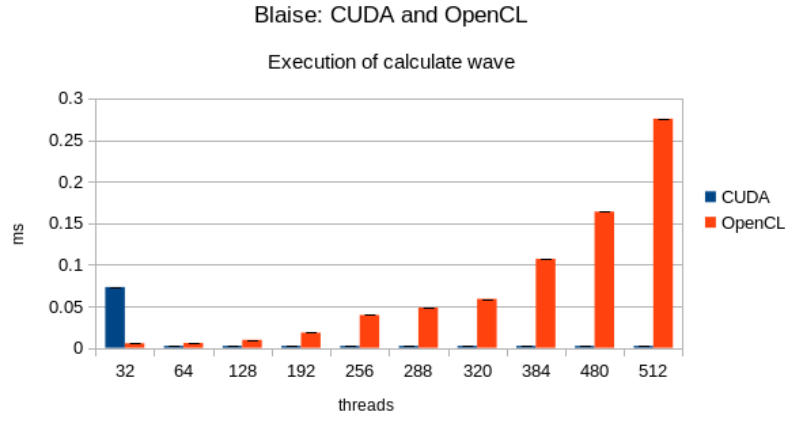


Figura 8: Tempo de execução do calculate_wave. Fonte: O autor

Somando todos os tempos de execução dos kernels, o resultado nos mostra que o melhor tempo de execução do CUDA é 66% mais rápido do que o OpenCL quando executado na Blaise.

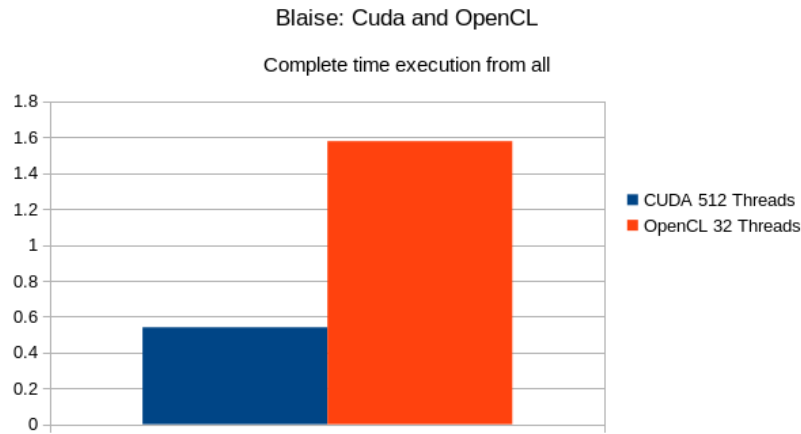


Figura 9: Tempo de execução somado. Fonte: O autor

5.2.1 Tabelas tempo de execução CUDA

Threads	Soma	Tempo Médio	Desvio Padrão
32	2.628905	0.0730251388	0.0000374165
64	0.08325	0.0023125	0.0000141421
128	0.083043	0.00230675	0.0000141421
192	0.083333	0.0023148055	0.00001
256	0.083299	0.0023138611	0.0000141421
288	0.083198	0.0023110555	0.0000141421
320	0.083232	0.002312	0
384	0.083216	0.0023115555	0.00001
480	0.082974	0.0023048333	0.0000244948
512	0.083317	0.0023143611	0.0000141421

Tabela 10: Tabela Tempo de execução da calculate_wave

Threads	Soma	Tempo Médio	Desvio Padrão
32	0.706816	0.0196337777	0.0002621068
64	0.082208	0.0022835555	0.0000818535
128	0.081888	0.0022746666	0.0000640312
192	0.08192	0.0022755555	0.00007
256	0.081536	0.0022648888	0.0000141421
288	0.081984	0.0022773333	0.0000608276
320	0.084384	0.002344	0.0003830143
384	0.081632	0.0022675555	0.0000608276
480	0.081984	0.0022773333	0.0001029563
512	0.082112	0.0022808888	0.0000781024

Tabela 11: Tabela Tempo de execução da define_c

Threads	Soma	Tempo Médio	Desvio Padrão
32	1.079424	0.029984	0.0001959591
64	0.0816	0.0022666666	0.000041231
128	0.081664	0.0022684444	0.0000632455
192	0.08176	0.0022711111	0.0000655743
256	0.082176	0.0022826666	0.0000812403
288	0.082528	0.0022924444	0.0001024695
320	0.08224	0.0022844444	0.0000806225
384	0.081888	0.0022746666	0.000067082
480	0.095584	0.0026551111	0.002230807
512	0.082144	0.0022817777	0.00008544

Tabela 12: Tabela Tempo de execução da define_initial_condition

Threads	Soma	Tempo Médio	Desvio Padrão
32	1.412352	0.039232	0.0088545186
64	0.114336	0.003176	0.0025073292
128	0.099136	0.0027537777	0.0001212435
192	0.115136	0.0031982222	0.0027657729
256	0.098112	0.0027253333	0.0000916515
288	0.116736	0.0032426666	0.0028729949
320	0.111648	0.0031013333	0.0023762786
384	0.113504	0.0031528888	0.0026491696
480	0.132032	0.0036675555	0.0035690895
512	0.127008	0.003528	0.0033541019

Tabela 13: Tabela Tempo de execução da initialize_u1

Threads	Soma	Tempo Médio	Desvio Padrão
32	0.683104	0.0189751111	0.000474763
64	0.083392	0.0023164444	0.0000860232
128	0.082144	0.0022817777	0.000072111
192	0.081504	0.002264	0.000034641
256	0.083424	0.0023173333	0.0001396424
288	0.083168	0.0023102222	0.0000894427
320	0.08224	0.0022844444	0.0000860232
384	0.083008	0.0023057777	0.0001144552
480	0.083456	0.0023182222	0.0001086278
512	0.082432	0.0022897777	0.0000883176

Tabela 14: Tabela Tempo de execução da initialize_u2

Threads	Soma	Tempo Médio	Desvio Padrão
32	0.677984	0.0188328888	0.0002273763
64	0.082208	0.0022835555	0.00007874
128	0.082176	0.0022826666	0.0000871779
192	0.081504	0.002264	0.00002
256	0.082528	0.0022924444	0.0000741619
288	0.09696	0.0026933333	0.0023212496
320	0.082112	0.0022808888	0.0000793725
384	0.082016	0.0022782222	0.000072111
480	0.082432	0.0022897777	0.000083666
512	0.083008	0.0023057777	0.0001048808

Tabela 15: Tabela Tempo de execução da initialize_u3

5.2.2 Tabelas tempo de execução OpenCL

Threads	Soma	Tempo Médio	Desvio Padrão
32	0.1916463777	0.0053235104	0.000069282
64	0.2086458695	0.0057957185	0.0000489897
128	0.3434376455	0.0095399345	0.0000141421
192	0.654563051	0.0181823069	0.0000264575
256	1.4183512352	0.0393986454	0.000041231
288	1.7401751055	0.0483381973	0.0000360555
320	2.1193291728	0.0588702548	0.0000282842
384	3.850758494	0.1069655137	0.0000223606
480	5.8960226003	0.1637784055	0.0000360555
512	9.9183282893	0.2755091191	0.0003333166

Tabela 16: Tabela Tempo de execução da calculate_wave

Threads	Soma	Tempo Médio	Desvio Padrão
32	0.228448	0.0063457777	0.0003038091
64	0.264032	0.0073342222	0.0025953419
128	0.3848	0.0106888888	0.0003906404
192	0.69168	0.0192133333	0.0003945883
256	1.46704	0.0407511111	0.0027702707
288	1.780352	0.0494542222	0.0002956349
320	2.162528	0.0600702222	0.0026366835
384	3.925568	0.1090435555	0.0027767246
480	6.000512	0.1666808888	0.0013820998
512	10.058496	0.2794026666	0.00042

Tabela 17: Tabela Tempo de execução da define_c

Threads	Soma	Tempo Médio	Desvio Padrão
32	0.204832	0.0056897777	0.0021080796
64	0.22432	0.0062311111	0.0023695358
128	0.359744	0.0099928888	0.0020189601
192	0.655264	0.0182017777	0.0001118033
256	1.420672	0.0394631111	0.000103923
288	1.747456	0.0485404444	0.0001486606
320	2.119168	0.0588657777	0.0001240967
384	3.88288	0.1078577777	0.0001191637
480	5.963904	0.165664	0.0001496662
512	10.034752	0.2787431111	0.0004850773

Tabela 18: Tabela Tempo de execução da define.initial_condition

Threads	Soma	Tempo Médio	Desvio Padrão
32	0.491904	0.013664	0.0032167841
64	0.507616	0.0141004444	0.0014846548
128	0.744928	0.0206924444	0.0060444271
192	0.933568	0.0259324444	0.0029337007
256	1.691456	0.0469848888	0.0028281089
288	2.000704	0.0555751111	0.0030969501
320	2.403136	0.0667537777	0.0029283271
384	4.166336	0.1157315555	0.0012417326
480	6.215104	0.1726417777	0.0011355615
512	10.279392	0.2855386666	0.0011889911

Tabela 19: Tabela Tempo de execução da initialize_u1

Threads	Soma	Tempo Médio	Desvio Padrão
32	0.239744	0.0066595555	0.0013034185
64	0.266016	0.0073893333	0.0029482537
128	0.385504	0.0107084444	0.0004395452
192	0.704032	0.0195564444	0.0026209349
256	1.45216	0.0403377777	0.0003148015
288	1.778752	0.0494097777	0.0003307567
320	2.142656	0.0595182222	0.0003280243
384	3.90592	0.1084977777	0.0003773592
480	5.9784	0.1660666666	0.000445982
512	10.043264	0.2789795555	0.0004643274

Tabela 20: Tabela Tempo de execução da initialize_u2

Threads	Soma	Tempo Médio	Desvio Padrão
32	0.220896	0.006136	0.0027037936
64	0.233344	0.0064817777	0.0025597851
128	0.354816	0.009856	0.0003337663
192	0.667072	0.0185297777	0.0005681549
256	1.42672	0.0396311111	0.0002576819
288	1.753312	0.0487031111	0.0002149418
320	2.11824	0.05884	0.0003074085
384	3.88144	0.1078177777	0.0003901281
480	5.955232	0.1654231111	0.000262488
512	10.020608	0.2783502222	0.0004049691

Tabela 21: Tabela Tempo de execução da initialize_u3

6 Análise PTX

Para compreender o comportamento do OpenCL, podemos analisar o PTX (Parallel Thread Execution) exibido abaixo do kernel `define_c`:

```
1 // .globl define_c
2 .entry define_c(
3 .param .u64 .ptr .global .align 8 define_c_param_0,
4 .param .f64 define_c_param_1,
5 .param .u32 define_c_param_2,
6 .param .u32 define_c_param_3
7 )
8 {
9 .reg .pred %p<4>;
10 .reg .b32 %r<16>;
11 .reg .f64 %fd<2>;
12 .reg .b64 %rd<4>;
13
14
15 ld.param.u64 %rd1, [define_c_param_0];
16 ld.param.f64 %fd1, [define_c_param_1];
17 ld.param.u32 %r3, [define_c_param_2];
18 ld.param.u32 %r4, [define_c_param_3];
19 mov.b32 %r5, %envreg3;
20 mov.u32 %r6, %ctaid.x;
21 mov.u32 %r7, %ntid.x;
22 mov.u32 %r8, %tid.x;
23 add.s32 %r9, %r8, %r5;
24 mad.lo.s32 %r1, %r7, %r6, %r9;
25 mov.u32 %r10, %ctaid.y;
26 mov.u32 %r11, %ntid.y;
27 mov.u32 %r12, %tid.y;
28 mov.b32 %r13, %envreg4;
29 add.s32 %r14, %r12, %r13;
30 mad.lo.s32 %r2, %r11, %r10, %r14;
31 setp.ge.s32 %p1, %r1, %r3;
32 setp.ge.s32 %p2, %r2, %r4;
33 or.pred %p3, %p1, %p2;
34 @%p3 bra $L__BB1_2;
35
36 mad.lo.s32 %r15, %r2, %r3, %r1;
37 mul.wide.s32 %rd2, %r15, 8;
38 add.s64 %rd3, %rd1, %rd2;
39 st.global.f64 [%rd3], %fd1;
40
41 $L__BB1_2:
42 ret;
43
44 }
```

Abaixo o PTX do código CUDA do kernel `define_c`:

```
1 // .globl _Z8define_cPddii
2 .visible .entry _Z8define_cPddii(
3 .param .u64 _Z8define_cPddii_param_0,
4 .param .f64 _Z8define_cPddii_param_1,
5 .param .u32 _Z8define_cPddii_param_2,
6 .param .u32 _Z8define_cPddii_param_3
7 )
8 {
9 .reg .pred %p<4>;
10 .reg .b32 %r<12>;
11 .reg .f64 %fd<2>;
12 .reg .b64 %rd<5>;
13
14
15 ld.param.u64 %rd1, [_Z8define_cPddii_param_0];
16 ld.param.f64 %fd1, [_Z8define_cPddii_param_1];
17 ld.param.u32 %r3, [_Z8define_cPddii_param_2];
18 ld.param.u32 %r4, [_Z8define_cPddii_param_3];
19 mov.u32 %r5, %tid.x;
20 mov.u32 %r6, %ntid.x;
21 mov.u32 %r7, %ctaid.x;
22 mad.lo.s32 %r1, %r7, %r6, %r5;
23 mov.u32 %r8, %ntid.y;
24 mov.u32 %r9, %ctaid.y;
25 mov.u32 %r10, %tid.y;
26 mad.lo.s32 %r2, %r9, %r8, %r10;
27 setp.ge.s32 %p1, %r1, %r3;
28 setp.ge.s32 %p2, %r2, %r4;
29 or.pred %p3, %p1, %p2;
30 @%p3 bra $L__BB1_2;
31
32 cvta.to.global.u64 %rd2, %rd1;
33 mad.lo.s32 %r11, %r2, %r3, %r1;
34 mul.wide.s32 %rd3, %r11, 8;
35 add.s64 %rd4, %rd2, %rd3;
36 st.global.f64 [%rd4], %fd1;
37
38 $L__BB1_2:
39 ret;
40
41 }
```

Observando o código, podemos notar que o OpenCL tem 8 instruções MOV e o CUDA tem 6, enquanto que o CUDA tem 17 instruções e o OpenCL tem 20, mas quando analisamos o código PTX inteiro disponível no [github\[2\]](#), temos o seguinte resultado para o código OpenCL:

Kernel Name	Flops (float int bit)	Copy	Branch
initialize_u	13 0 10 3	12	1
define_c	13 0 10 3	12	1
define_initial_condition	65 33 20 12	39	3
calculate_wave	48 17 23 8	16	1

O número de memória cache zerado mostra que não foi utilizado memória compartilhada, o que poderia ser uma otimização válida na desenvolvimento do código, tanto em OpenCL quanto em CUDA.

Kernel Name	Cache (load store)	Memory (load cached store)
initialize_u	0 0 0	8 0 0 8
define_c	0 0 0	8 0 0 8
define_initial_condition	0 0 0	8 0 0 8
calculate_wave	0 0 0	64 56 0 8

No código CUDA, a análise do código PTX tem o seguinte resultado:

Kernel Name	Flops (float int bit)	Copy	Branch
_Z12initialize_uPdii	11 0 8 3	11	1
_Z8define_cPdii	11 0 8 3	11	1
_Z24define_initial_conditionPdiiii	63 33 18 12	38	3
_Z14calculate_wavePdS_S_S.iidd	43 14 21 8	18	1

Kernel Name	Cache (load store)	Memory (load cached store)
_Z12initialize_uPdii	0 0 0	8 0 0 8
_Z8define_cPdii	0 0 0	8 0 0 8
_Z24define_initial_conditionPdiiii	0 0 0	8 0 0 8
_Z14calculate_wavePdS_S_S.iidd	0 0 0	64 56 0 8

Podemos notar que a quantidade de operações de ponto flutuante no OpenCL é significativamente maior e o número de cópias também, algo que reflete a falta de acesso a mais registradores de 64bits no OpenCL, o que também acaba prejudicando operações de ponto flutuante de precisão dupla, que foi utilizado no simulador.

6.1 Conclusões

O desempenho do CUDA, tanto com controle ou sem controle do Clock, é notoriamente superior ao do OpenCL em placas Nvidia, alcançando um desempenho melhor em placas AMD, porém o desenvolvimento do simulador foi realizado sem utilizar nenhuma otimização específica, algo que o CUDA já tem embutido em seu desenvolvimento e fica explícito quando observamos seu código PTX, pois tem acesso a mais registradores, realiza menos operações de ponto flutuante e inteiros, além disso tem menos operações realizadas em geral, como podemos observar no código de exemplo do kernel define_c em PTX.

Referências

- [1] Maksim Baz. Acoustic wave simulation in cuda. https://github.com/maksimbaz/acoustic_wave_cuda, 2024. Accessed: 2024-08-30.
- [2] Lucas Villela Canoas. Acoustic waves with cuda and opencl. https://github.com/lcsvillela/acoustic_waves_opencl_and_cuda, 2024. Accessed: 2024-08-30.
- [3] Amanda Sabatini Dufek. Introdução ao opencl. <http://www.cenapad-rj.lncc.br/tutoriais/materiais-hpc/semana-sdumont/opencl.pdf>, 2024. Accessed: 2024-08-30.
- [4] Rodrigo Fernandes Guimarães. Modelador de propagação de onda acústica 2d por diferenças finitas no domínio do tempo em cuda. Master's thesis, Brasil, 2018.
- [5] Vosen. Zluda. <https://github.com/vosen/ZLUDA>, 2024. Accessed: 2024-08-30.